

On Batch Writing in COTS RFID Systems

Kanghuai Liu¹, Lin Chen², *Member, IEEE*, Jihong Yu³, *Member, IEEE*, and Haochen Cui⁴

Abstract—We study the batch writing problem in RFID systems, where the reader seeks the most time-efficient way to write information into a given subset of tags. The problem is analogous to the multicast problem in classical networks, but one-to-many transmission is not supported in COTS RFID systems. Driven by the technical challenge, this paper addresses the problem of designing batch writing algorithms for COTS RFID systems. We make three contributions. First, we establish the minimal execution time for any batch writing algorithm, thus setting the theoretical performance limit. Second, we quantitatively compare and gauge the existing propositions applicable to our problem. Third, we develop a novel batch writing algorithm with minimal 25% performance gain over the best state-of-the-art solution. Our key technicalities are designing an encoding scheme allowing the reader to efficiently perform batch writing and optimizing the batch writing sequence to minimize the overall execution time. We also perform extensive experiments to demonstrate the effectiveness of our algorithm.

Index Terms—Batch writing, throughput optimization, C1G2 standard, COTS RFID systems.

I. INTRODUCTION

DISTINGUISHED by its low cost and high energy efficiency, radio frequency identification (RFID) offers a viable solution for the Internet of Things (IoT) [1]. A standard RFID system contains a back-end server, a reader, and a large number of tags attached to the objects. The reader, connected to the back-end server with powerful storage and computing capabilities, interrogates tags by transmitting radio frequency (RF) signals to them, while tags reply by backscattering the received RF signals [2].

In this paper, we study a fundamental problem in RFID systems: how the reader writes information into a given subset of tags in the most time-efficient way. Formally, suppose that, in an RFID network composed of a set $\mathcal{N}$ of $N \triangleq |\mathcal{N}|$ tags, the reader needs to write a piece of information to a subset $\mathcal{T} \subseteq \mathcal{N}$ of $T \triangleq |\mathcal{T}|$ tags, we seek an algorithm minimizing the expected time to complete the entire writing task. We term this problem

as the *batch writing* problem. Batch writing arises in a variety of practical applications, in which the reader needs to load common information to multiple tags. For instance, in supermarkets, the salesman needs to update inventory information to the newly released products, among which those belonging to the same shelf can be transacted as a batch. He can thus write the shelf number to the products batch by batch. As another example, the emerging technique of over-the-air programming in RFID systems requires tags belonging to a same functional category to be loaded the same data and code [3]. In both cases, increasing the batch writing throughput is of vital importance.

A naive approach [4] to batch writing is to poll each tag one by one and write the information into the tag, but this approach is clearly inefficient by neglecting the optimization opportunity, where some tags can be selected in a single batch. An alternative solution [5] is to attribute a common group ID to each tag in $\mathcal{T}$ and later multicast the information via the pre-loaded group ID. This approach seems to offer an efficient solution, but it has two critical limitations. First, the target tag set $\mathcal{T}$ may be dynamic and thus cannot be determined *a priori*, making it impossible to pre-load the group ID. Second, the one-to-many transmission, classical in standard networks, is not supported in Class1 Gen2 (C1G2) [6] standard, making the group-based multicast incompatible with the commodity off-the-shelf (COTS) passive RFID tags.

Driven by the above observation, we embark in this paper on a comprehensive analysis of the batch writing problem by answering the following research questions.

- Q1:** What is the theoretical performance limit of batch writing algorithms? How large is the performance gap between the performance limit and the state-of-the-art solutions?
- Q2:** How to design efficient batch writing algorithms approaching the theoretical performance limit and compatible with COTS RFID tags?

To answer Q1, we mobilize theoretical tools in communication and computation complexity to analytically establish the expected execution time lower-bound for *any* batch writing algorithm. Technically, we reduce the 2-NEQ problem to a sub-problem of our batch writing problem via Cook reduction [7], and then leverage the tiling method [8] to derive the lower-bound of the execution time for any batch writing algorithm. We note that the theoretical performance limit of batch writing has not been formally characterized before. Our result derived in this paper fills this void and provides performance reference for future research in this field. Armed with the performance limit, we further quantitatively compare and gauge the existing propositions applicable to our problem (cf. Table III).

Manuscript received 18 February 2023; revised 22 April 2023; accepted 29 May 2023. Date of publication 6 June 2023; date of current version 4 April 2024. The work of Lin Chen was supported in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2022A1515012309 and in part by Pearl River Talent Program under Grant 2019QN01X140. The work of Jihong Yu was supported in part by the NSFC under Grant 61901035. Recommended for acceptance by Z. Li. (*Corresponding author: Lin Chen.*)

Kanghuai Liu, Lin Chen, and Haochen Cui are with the School of Computer Science and Engineering, Guangdong Province Key Laboratory of Information Security Technology, Sun Yat-sen University, Guangzhou 510275, China (e-mail: iukh8@mail2.sysu.edu.cn; chenlin69@mail.sysu.edu.cn; cuihch5@mail2.sysu.edu.cn).

Jihong Yu is with the School of Information and Electronics, Beijing Institute of Technology, Beijing 100811, China (e-mail: jihong-yu@hotmail.com).

Digital Object Identifier 10.1109/TMC.2023.3283238

TABLE I
MAIN NOTATIONS

$\mathcal{N}$	Tag set
N	Cardinality of $\mathcal{N}$, $N = \mathcal{N} $
$\mathcal{T}$	Set of target tags, $\mathcal{T} \subseteq \mathcal{N}$
T	Number of target tags, $T = \mathcal{T} $
$\mathbb{A}, \mathbb{B}$	Inventoried flags of tags
L	Tag pseudo-ID length
B	Number of tag buckets/groups
$\mathcal{B}_b$	b -th tag bucket
$\mathcal{T}_b$	Target tags belonging to $\mathcal{B}_b$, $\mathcal{T}_b \triangleq \mathcal{B}_b \cap \mathcal{T}$
n	Size of each bucket, $n \triangleq N/B$
m	Number of MASKS within a SELECT command
k	Length of discrete bundling code (DBC)
q	Number of 01's within a DBC
δ	Number of bits for data storage
S	Number of SELECT commands required
Λ	Overall execution time of each algorithm
R	Ratio of S in CTB to that in other algorithms

TABLE II
TAG RESPONSE TO DIFFERENT ACTIONS IN SELECT

Action	Target tags	Non-target tags
000	inventoried flag $\rightarrow \mathbb{A}$	inventoried flag $\rightarrow \mathbb{B}$
001	inventoried flag $\rightarrow \mathbb{A}$	do nothing
010	do nothing	inventoried flag $\rightarrow \mathbb{B}$
011	$\mathbb{A} \rightarrow \mathbb{B}$ or $\mathbb{B} \rightarrow \mathbb{A}$	do nothing
100	inventoried flag $\rightarrow \mathbb{B}$	inventoried flag $\rightarrow \mathbb{A}$
101	inventoried flag $\rightarrow \mathbb{B}$	do nothing
110	do nothing	inventoried flag $\rightarrow \mathbb{A}$
111	do nothing	$\mathbb{A} \rightarrow \mathbb{B}$ or $\mathbb{B} \rightarrow \mathbb{A}$

TABLE III
COMPARISON OF EXISTING BATCH WRITING ALGORITHMS

Algorithm	Expected execution time
EW [4]	T
WB, GWB [5]	$\frac{T(N-T+B)}{N} + \frac{T(B-1)}{N}$
CTB (our solution)	$\frac{3T(N-T+B)}{4N} + \frac{T(B-1)}{N}$
Theoretical performance limit	$\frac{\min\{T, N-T\}}{16 \log L}$

To answer Q2, we develop a novel batch writing algorithm, termed as compact tag bundling (CTB), and prove that CTB outperforms the best state-of-the-art proposition by at least 25%. The central novelty in our design is two-fold. First, we design an encoding scheme, termed as bundling vector (BV), allowing the reader to efficiently perform batch writing with a minimum number of SELECT commands. Second, based on the encoding scheme, we optimize the batch writing sequence by exploiting the functionality offered by the Class1 Gen2 (C1G2) standard to make an optimal decision among selecting target tags, deselecting the non-target tags, and combining the two strategies, with the objective of minimizing the overall time overhead. We then theoretically establish the performance of CTB and prove that it outperforms the best state-of-the-art proposition by at least 25%, advancing the state of the art on batch tag writing. Nevertheless, compared to the derived performance limit, there is still room for improvement over our algorithm. We hope that

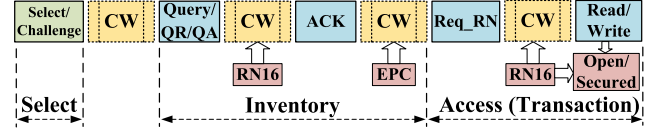


Fig. 1. A writing cycle in C1G2. QUERY, QUERYREP and QUERYADJ are used to initialize a new frame, among which QUERYREP and QUERYADJ are broadcast by the reader in tag identification. ACK, CW, RN16, and REQ_RN are the acknowledge command, the continuous wave, the 16-bit random number and the request RN16 command, respectively.

our work will stimulate further study that can narrow or even fill the performance gap.

Roadmap: The rest of the paper is articulated as follows. Section II introduces the technical background and optimization objective. Theoretical performance limit is then derived in Section III. Section IV quantifies major existing batch writing algorithms applicable to our problem. We present our solution CTB and evaluate its theoretical performance in Section V. Extensive experiments are conducted in Section VI. Section VII summarizes related work and analyzes their limitations. Section VIII concludes the paper.

II. TECHNICAL BACKGROUND AND PROBLEM FORMULATION

In the C1G2 standard, tag writing is realized by the reader via a so-called *writing cycle* composed of three successive operations: SELECT, INVENTORY, and ACCESS. SELECT chooses a subset of tags that will participate in the subsequent steps. INVENTORY then executes an inventory round to identify the selected tags. ACCESS further writes the information to the identified tags one by one. At the protocol level shown in Fig. 1, after the SELECT operation, the reader initializes a frame or a slot by broadcasting QUERY or QR commands, respectively. With the received QUERY command, each selected tag generates a 16-bit random number RN16, and transmits it to the reader. Upon receiving a legal RN16 from a tag, the reader acknowledges the RN16 by returning an ACK command to this tag. After that, this tag replies its EPC to the reader. Followed by the INVENTORY operation, the reader first queries the RN16 again by transmitting the REQ_RN command, then dialogues with it by performing read, write or other transactions [6].

The key technicality to optimize the efficiency of batch writing hinges on decreasing the time overhead of SELECT. The reason is three-fold.

- First, a clever design of SELECT can exploit the full potential of batching, thus minimizing the number of writing cycles and hence the writing time.
- Second, [5] has conducted comprehensive experiments by writing 16-bit information into each tags' memory, empirically demonstrating that SELECT takes over 60% of the total execution time of a writing cycle.
- Third, in the C1G2 standard, INVENTORY and ACCESS are mandatory and non-modifiable: the reader is only permitted to perform the inventory-write procedure on tags one by one, which makes it impossible to reduce the time overhead caused by INVENTORY and ACCESS. By contrast, SELECT

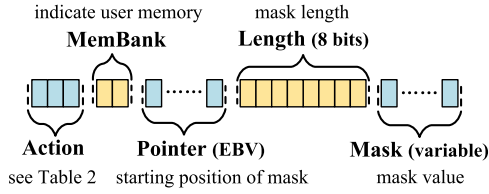


Fig. 2. Illustration of the SELECT command. Each yellow or blue rectangle represents one-bit length.

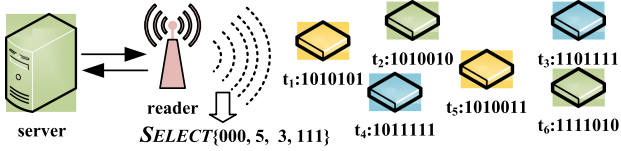


Fig. 3. Illustration of SELECT in C1G2 standard.

is optionally configurable, offering us technical support to decrease its time cost and hence improve the overall batch writing efficiency.

To shed more light on the batch writing problem and our design rationale, we need to look into more detail on how SELECT operates in current C1G2 communication standard.

A. SELECT Command in C1G2 Standard

SELECT operation in the C1G2 standard defines a command SELECT, allowing the reader to select a set of tags based on user-defined criteria [6] via the following parameters: ACTION, MEMBANK, POINTER, LENGTH, MASK, as demonstrated in Fig. 2. Technically, MEMBANK specifies the tag memory bank for comparison. For notation convenience, we refer to the content in MEMBANK of each tag as the *pseudo-ID* of the tag in this paper. POINTER and LENGTH further specify a starting bit in tag pseudo-ID to compare with a mask, and the length of this mask, respectively. MASK then specifies a mask to be applied to POINTER and LENGTH.

With the above four parameters, the reader can divide tags into two categories, those with a pseudo-ID matching the specified mask are termed as *target tags*, while the rest are termed as *non-target tags*. The reader can then use ACTION to impose different actions to target tags and non-target tags, notably setting their inventoried flags, as in Table II. By setting the inventoried flags of target tags to $\mathbb{A}$ or $\mathbb{B}$ while the others to the opposite, the reader can select the target tags to participate in the subsequent INVENTORY and ACCESS operations while filtering out the non-target tags. By combining the above analysis, the SELECT command can be abbreviated as

$$\text{SELECT}\{\underbrace{3 \text{ bits}}_{\text{ACTION}}, \underbrace{\text{EBV}}_{\text{POINTER}}, \underbrace{8 \text{ bits}}_{\text{LENGTH}}, \underbrace{\text{variable}}_{\text{MASK}}\}.$$

To further clarify SELECT, we give an illustrative example in Fig. 3, where $\mathcal{T} = \{t_3, t_4\}$. To select $\mathcal{T}$, the reader sends $\text{SELECT}\{000, 5, 3, 111\}$. On receiving the SELECT command, each tag compares the 3-bit substring starting from the 5th bit in

its own pseudo-ID with the MASK 111. With the ACTION 000, the target tags t_3 and t_4 then set their inventoried flags to $\mathbb{A}$ based on Table II, while the non-target tags t_1, t_2, t_5, t_6 set their inventoried flags to $\mathbb{B}$. Hence, the reader can select $\mathcal{T}$ by only allowing the tags with an inventoried flag $\mathbb{A}$ to participate in the current writing cycle.

B. Optimal Batch Writing Problem Formulation

By examining the above procedure of the writing cycle in C1G2 standard, we can readily observe that minimizing the batch writing time can be mapped to minimizing the number of SELECT commands required to select all the target tags. Our batch writing problem can thus be formulated as constructing a pseudo-ID for each tag and developing a batch writing algorithm to minimize the number of SELECT commands.

III. ALGORITHM-INDEPENDENT PERFORMANCE BOUND

In this section, we establish the lower-bound of the expected number of SELECT commands of *any* batch writing algorithm, thus setting the theoretical performance limit. Technically, we first reduce the 2-NEQ problem [9] to a subproblem of our problem via Cook reduction [7], then leverage the tiling method [10] to establish the performance limit in Theorem 1.

Technically, we let two players Alice and Bob virtually operate at the reader and tag sides to execute a batch writing protocol $\mathcal{P}$ to select a subset $\mathcal{T}$ of T tags from the entire tag set $\mathcal{N} = \{t_i\}_{i=1}^N$, each characterized by a unique L -bit pseudo-ID ($L \geq \log N$).¹ Specifically, Alice transmits a set of SELECT commands to Bob by encoding the pseudo-IDs of the tags in $\mathcal{T}$ in the commands. Bob compares the masks in the received SELECT commands with the pseudo-IDs to change the inventory flags in the tags accordingly. We first give an auxiliary lemma.

Lemma 1: At least $\min\{T, N - T\}$ comparisons are necessary to select all the tags in $\mathcal{T}$.

The proof is intuitive by noticing that (1) if $T < N/2$, the luckiest case corresponds to the situation where each of the first T comparisons performed by Bob incurs a match; (2) if $T \geq N/2$, the luckiest case is the situation where none of the first $N - T$ comparisons performed by Bob incurs a match.

We next zoom on a single comparison operation allowing Bob to verify if a tag incurs a match or not, termed as the *atomic comparison* problem. Technically, we establish a Cook reduction from the famous 2-NEQ problem [9] to our atomic comparison problem. The Cook reduction states that a problem $\mathcal{A}$ can be reduced to another problem $\mathcal{B}$ iff there exists a polynomial-time oracle machine that can invoke every solution to $\mathcal{B}$ as a subroutine to solve $\mathcal{A}$. This implies that $\mathcal{B}$ is at least as hard as $\mathcal{A}$ and hence a lower-bound on the time overhead of any solution to $\mathcal{B}$ can be derived from that to $\mathcal{A}$. In our reduction, given atomic comparison as $\mathcal{B}$, we choose 2-NEQ as $\mathcal{A}$. 2-NEQ is a two-party communication game where two players with limited computational power hold two Q -bit binary strings $x, y \in \{0, 1\}^Q$ as their inputs, respectively. They collaboratively compute z , where $z = 1$ if $x = y$, 0 otherwise. Each player is

¹Throughout this paper, logarithm is taken on base 2.

unaware of the input held by the other player. Therefore, they need to build a communication protocol $\mathcal{P}_A$ to exchange their information so as to execute the task. We define the *cost* of $\mathcal{P}_A$, denoted by $C(\mathcal{P}_A)$, as the expected number of bits exchanged between the two players. A theoretical result from [9] gives a lower-bound on $C(\mathcal{P}_A)$, as stated in Lemma 2.

Lemma 2: For any $\mathcal{P}_A$, $C(\mathcal{P}_A) \geq \log Q$.

To Cook-reduce 2-NEQ to atomic comparison, given any atomic comparison protocol $\mathcal{P}_B$, we use it as a building block to design an algorithm $\mathcal{P}_A$ to solve 2-NEQ. To this end, it suffices to notice that verifying a matching between a tag and a MASK in a SELECT command can be mapped to checking whether the corresponding two sequences are equal and hence to solving a 2-NEQ instance. The detailed demonstration of the reduction is given in Lemma 3.

Lemma 3: Any instance of 2-NEQ can be cook-reduced to the atomic comparison problem.

Proof: We prove that, given any atomic comparison protocol $\mathcal{P}_B$, we can use it as a building block to design an algorithm $\mathcal{P}_A$ to solve 2-NEQ. To this end, we design the following algorithm that constructs $\mathcal{P}_A$ from $\mathcal{P}_B$. The construction is composed of three steps. We denote by I_i the unique ID of tag t_i .

Step 1: Initialization: Since Alice and Bob virtually operate at the reader and tag sides, respectively, they then can be seen as the two players in the 2-NEQ problem. The corresponding MASK transmitted from Alice and I_i held by Bob are regarded as the two Q -bit binary strings $x, y \in \{0, 1\}^Q$ from the two players in 2-NEQ problem, respectively, which strictly makes the length of I_i or MASK equal Q .

Step 2: Execution: By executing $\mathcal{P}_B$ on t_i , the two players in 2-NEQ problem can readily determine t_i is selected by $\mathcal{P}_B$ or not. Specifically, t_i is selected iff $\text{MASK} \triangleq I_i$, and not-selected otherwise.

Step 3: Output Conversion: Eventually, we convert the output from $\mathcal{P}_B$ to that from $\mathcal{P}_A$. In Step 2, the event that t_i is selected indicates $\text{MASK} \triangleq I_i$, further implying $x = y$ and hence $z = 1$. Conversely, the event that t_i is not-selected reveals $z = 0$. This is equivalent to that z can be successfully computed as long as t_i is correctly marked.

The above reduction is valid because (1) it can be completed within polynomial time, (2) the 2-NEQ problem is solved by $\mathcal{P}_A$ once the atomic comparison problem is solved by $\mathcal{P}_B$. As a consequence, the Cook-reduction from the 2-NEQ problem to our atomic comparison problem is proved by construction. $\square$

As each tag is encoded by L bits, it follows from Lemma 2 that

$$C(\mathcal{P}_B) \geq C(\mathcal{P}_A) \geq \log L. \quad (1)$$

By combining Lemma 1 and (1), we can derive the minimal number of bits for any batch writing protocol $\mathcal{P}$ to successfully accomplish its task, i.e., its cost, as follows

$$C(\mathcal{P}) \geq \min\{T, N - T\} \log L.$$

Moreover, the length of a MASK is at most L because it is a substring of the L -bit tag pseudo-ID. Therefore, $\mathcal{P}$ thus needs at

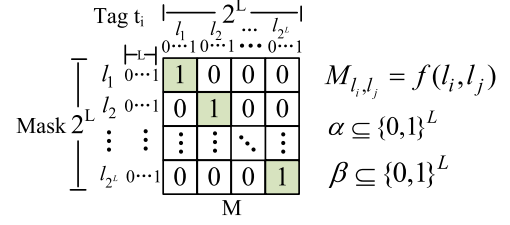


Fig. 4. Illustration of boolean matrix for atomic comparison.

least

$$\frac{\min\{T, N - T\} \log L}{L} = \frac{\min\{T, N - T\}}{L / \log L}.$$

SELECT commands to select $\mathcal{T}$ from $\mathcal{N}$.

Remark: For presentation conciseness, in this paper we do not consider aggregation of SELECT commands, where m MASKS can be aggregated in a single SELECT. It suffices to divide the result by m to take into account this.

The above lower-bound can be further improved. In the rest of this section, we leverage the concept of monochromatic tiling to derive a tighter bound. The readers may need to refer to [8] for more detailed technical background.

Definition 1 (Monochromatic rectangle [8]): Given a boolean matrix $M = \{f(i, j) | i \in \{0, 1\}^g, j \in \{0, 1\}^g\}$ for function $f : \{0, 1\}^g \times \{0, 1\}^g \rightarrow \{0, 1\}$, a monochromatic rectangle $\mathcal{R}$ is a submatrix of M corresponding to $\alpha \times \beta$ where $\alpha \subseteq \{0, 1\}^g$ and $\beta \subseteq \{0, 1\}^g$ and for $i \in \alpha$ and $j \in \beta$, $f(i, j)$ is the same.

Definition 2 (Monochromatic tiling [8]): A monochromatic tiling of M is a partition of M into disjoint monochromatic rectangles.

Lemma 4 (Tiling bound [8]): For any protocol $\mathcal{P}_f$ correctly computing f , it holds that $\log h \leq C(\mathcal{P}_f) \leq 16(\log h)^2$, where h is the minimal number of rectangles in any monochromatic tiling of M .

To analyze the maximum cost of a SELECT command, we consider the worse case where the length of a MASK equals L . We first treat the atomic comparison on a tag t_i as f where $g = L$. By so, the L -bit MASK l_j sent by Alice to select t_i and the pseudo-ID l_i of t_i held by Bob are regarded as two inputs of f . $f(l_i, l_j) = 1$ iff $l_i = l_j$ and 0 otherwise. To mark t_i , Bob then needs to compute f correctly. By Definitions 1 and 2, he designs a boolean matrix M containing $2^L \times 2^L$ blocks to compute f , as illustrated in Fig. 4, where the 2^L distinct inputs for MASK/ t_i are mapped to 2^L rows/columns and $M_{l_i, l_j} = f(l_i, l_j)$. To compute f , Bob with $\mathcal{P}_B$ takes a number of bits to verify if M_{l_i, l_j} equals 1. Specifically, he checks the blocks along the diagonal of M one by one and if M_{l_i, l_j} occurs among them. Each check partitions M into distinct monochromatic rectangles. As the lengths of l_i and l_j are L , Bob then needs at most L checks to compute f , leading to a monochromatic tiling containing $L \times L$ blocks. With L 1's along its diagonal, the tiling has at least L monochromatic rectangles. By Lemma 4, we obtain that Bob with $\mathcal{P}_B$ takes at most $16(\log L)^2$ bits to correctly compute f and hence marks t_i . As $\mathcal{P}_B$ needs at least one SELECT command to select t_i , each SELECT then consumes at most $16(\log L)^2$ bits. With an overall

overhead of $\min\{T, N - T\} \log L$, $\mathcal{P}$ thus needs at least

$$\frac{\min\{T, N - T\} \log L}{16(\log L)^2} = \frac{\min\{T, N - T\}}{16 \log L}.$$

SELECT commands to select $\mathcal{T}$ from $\mathcal{N}$. Combining the above analysis leads to a tighter theoretical performance lower-bound $\frac{\min\{T, N - T\}}{16 \log L}$, as summarized in Theorem 1.

Theorem 1 (Algorithm-independent performance bound): Any batch writing algorithm needs at least $\frac{\min\{T, N - T\}}{16 \log L}$ SELECT commands to select $\mathcal{T}$ from $\mathcal{N}$.

We conclude this section by remarking the trade-off between the time and space overheads implied by Theorem 1, where the space overhead corresponds to the L -bit pseudo-ID stocked at each tag.

IV. EXISTING BATCH WRITING ALGORITHM ANALYSIS

Given the performance limit established in Theorem 1, in this section we quantify the major existing batch writing algorithms. To our knowledge, the state-of-the-art algorithms applicable to the batch writing problem include exclusive writing (EW) [4], write bundling (WB) [5], and generalized write bundling (GWB) [5], which we analyze in the following and recapture our results in Table III.

EW has two key steps, selecting a tag by broadcasting its tag ID, and writing the corresponding information to the tag. Given $\mathcal{T}$ of T tags, EW requires T SELECT commands accordingly.

In WB, a group of consecutive tags are called a tag run.² The number of required SELECT commands, denoted by R , equals the number of tag runs in $\mathcal{T}$. It is derived in [5] that

$$\Pr[R = r] = \binom{N - T + 1}{r} \binom{T - 1}{T - r} / \binom{N}{T}. \quad (2)$$

It follows from (2) that R follows the hypergeometric distribution $R \sim \mathbb{H}(N, T, T - 1)$ [11]. By computing its expectation, we have $\mathbb{E}[R] = T(N - T + 1)/N$.

GWB is generalized from WB. It divides the pseudo-ID into two parts, a group ID and an index with the group, so as to make the pseudo-ID fit into the limited tag memory. Suppose that tags in $\mathcal{T}$ are distributed uniformly among $\mathcal{N}$, and the tag cardinality $\mathcal{N}$ is divided into B groups $\{g_i\}_{i=1}^B$, each containing N/B tags, among which T/B tags are target ones. WB then can be regarded as GWB with only one group. Let define R_i and R_w as the numbers of SELECT commands required in the i th group g_i and all the B groups, respectively. By mapping WB to single group in GWB, we then have

$$\mathbb{E}[R_i] = \frac{T/B(N/B - T/B + 1)}{N/B} = \frac{T(N/B - T/B + 1)}{N}.$$

Since GWB performs the entire writing task group by group, each group is independent of the others. By applying Wald's equality [12] and the law of large numbers [13] to generalize g_i

to all the B groups, we further obtain

$$\mathbb{E}[R_w] = \mathbb{E}\left[\sum_{i=1}^B R_i\right] = B \cdot \mathbb{E}[R_i] = \frac{T(N - T + B)}{N}. \quad (3)$$

Besides, since each group contains N/B tags, then T tags in $\mathcal{T}$ cover TB/N groups on average. GWB thus takes extra TB/N SELECT commands to eliminate the interference among the TB/N groups, in which a command can select only one group while deselecting others. But since the last group to be selected is not interfered by other groups, the additional overhead is then reduced to $T(B - 1)/N$. With (3), on expectation, we thus obtain the overall overhead of GWB as $T(N - T + B)/N + T(B - 1)/N$. Globally, the number of SELECT commands needed by WB/GWB can be summarized as

$$\frac{T(N - T + B)}{N} + \frac{T(B - 1)}{N},$$

with $B = 1$ for WB.

Our analysis on existing batch writing solutions indicates that there is still room for improvement over the existing propositions in reducing the time overhead.

V. BATCH WRITING BASED ON COMPACT TAG BUNDLING

Driven by the theoretical performance limit established in Section III, we develop a novel batch writing algorithm, termed as Compact Tag Bundling (CTB) outperforming all the propositions in the literature.

The key innovation of our algorithm is two-fold.

- *Pseudo-ID design:* We attribute a unique pseudo-ID to each tag, termed as bundling vector (BV), allowing the reader to efficiently perform batch writing with a minimum number of SELECT commands. A BV is composed of three segments: bucket ID (BID), contiguous bundling code (CBC), and discrete bundling code (DBC). BID allows to select a subset of tags in a single bucket. CBC then enables the reader to bundle contiguous tags with a number of contiguous 0's and 1's, while DBC further allows discrete bundling. The holistic combination of the three segments can facilitate the optimization of masks to cover tags.
- *Batch writing sequence optimization:* Armed with the above BV, we further optimize the batch writing sequence by exploiting the ACTION parameters specified in Table II to make an optimal decision among selecting target tags, deselecting the non-target tags, and combining the two strategies, with the objective of minimizing the number of SELECT commands, hence further reducing the time overhead.

In what follows, we first present the design of BV, then our batch writing algorithm and the optimization strategy, followed by the theoretical performance analysis.

A. Tag Pseudo-ID Design: Bundling Vector

We construct a unique pseudo-ID to each tag, called bundling vector (BV), composed of three segments, as demonstrated in Fig. 5.

²We refer the readers to [5] for detailed description of WB.

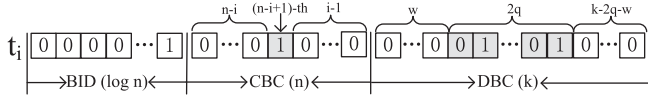


Fig. 5. Illustration of tag pseudo-ID design.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
t_1 :	0	0	1	0	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0
t_2 :	0	0	1	0	0	0	0	0	0	1	0	0	0	1	0	1	0	1	0
t_3 :	0	0	1	0	0	0	0	0	1	0	0	0	0	1	0	1	0	1	0
t_4 :	0	0	1	0	0	0	0	1	0	0	0	1	0	0	0	1	0	1	0
t_5 :	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0	0	1	0	0
t_6 :	0	0	1	0	0	1	0	0	0	0	0	1	0	1	0	0	0	1	0
t_7 :	0	0	1	0	1	0	0	0	0	0	0	0	1	0	1	0	0	0	0
t_8 :	0	0	1	1	0	0	0	0	0	0	0	1	0	1	0	1	0	0	0

Fig. 6. BVs of tags in bucket $\mathcal{B}_b$ under $n = 8$, $q = 3$, $k = 8$, and $BID = 001$.

Bucket ID (BID): We divide the tag set $\mathcal{N}$ into B disjoint buckets $\{\mathcal{B}_b\}_{b=1}^B$ such that each bucket contains $n \triangleq N/B$ tags.³ The first segment in the BV of each tag contains a $\log n$ -bit bucket ID. BID allows to select the tags in a single bucket or filter out the tags in other buckets. For instance, as illustrated in Fig. 6, the reader can select the tags in bucket $\mathcal{B}_b$ with $\text{SELECT}\{000, 1, 3, 001\}$, as depicted in the red box.

Contiguous Bundling Code (CBC): CBC is encoded as an n -bit vector, in which the $(n - i + 1)$ th bit of t_i is set to 1 while the others to 0, as illustrated in Fig. 5. CBC enables the reader to bundle contiguous tags with a single mask of contiguous 0's. This can be illustrated by Fig. 6. To select $\{t_3, t_4, t_5\}$, the reader broadcasts $\text{SELECT}\{010, 7, 3, 000\}$. As only $\{t_3, t_4, t_5\}$ do not match the mask, as illustrated by the green boxes, their inventoried flags are set to $\mathbb{B}$ based on ACTION 010, while the others do nothing. As a result, $\{t_3, t_4, t_5\}$ are selected in the current writing cycle while the others are filtered out.

Discrete Bundling Code (DBC): As described above, CBC allows the reader to select a set of contiguous tags, e.g., $\{t_3, t_4, t_5\}$ in Fig. 6. DBC, on the other hand, allows to select a set of discrete tags. Technically, DBC is formalized as a k -bit vector, in which a $2q$ -bit substring composed of q 01's cyclically rotates from one tag to another, as illustrated in Fig. 5 by the grey shadow. The rationale of the DBC design is to have an alternate distribution between 0 and 1 in each column of the matrix corresponding to the DBC part, hence enabling the reader to select or deselect discrete tags. For example, the 1's in the blue boxes of Fig. 6 allows the reader to select $\{t_2, t_4, t_8\}$ with a single command $\text{SELECT}\{000, 16, 1, 1\}$. As another example, the reader can select $\{t_1, t_2, t_4, t_6, t_8\}$ with only one command $\text{SELECT}\{010, 19, 1, 1\}$ that filters out $\{t_3, t_5, t_7\}$, as illustrated in the purple boxes of Fig. 6.

³Buckets in our design is similar to groups in GWB. We argue that the former is more appropriate as tags within a same group usually share certain common property, e.g., belonging to a same category.

B. Batch Writing Algorithm

We first demonstrate the pseudo-ID pre-loading operation and then our CTB algorithm design.

1) **Pseudo-ID Pre-Loading:** Before executing the algorithm, we construct and then pre-load the BV to each tag. The pre-loading operation is widely adopted in existing tag monitoring algorithms [5], [14], [15] on COTS RFID systems. It is also supported by main-stream COTS tags, such as Impinj [16] and Alien [17] tags, without any software or hardware modification. Technically, the SELECT can be applied in the original ID, EPC or user memory of these tags [16], [17]. As the original ID is non-modifiable and it may pose potential security risks to modify the EPC, we then pre-load the BV into the user memory. Alternatively, users can also ask the RFID manufacturer to write the BV into tags' user memory while manufacturing them [16], [17]. In either case, as the BV can be reused, the pre-loading operation only needs to be performed once. The same BV can be reused in the subsequent batch writing operations. Therefore, the cost related to loading BV is amortized.

2) **CTB Algorithm Design:** Armed with the BV design, we further optimize the batch writing sequence by exploiting the ACTION parameters specified in Table II to make an optimal decision among selecting target tags, deselecting the non-target tags, and combining the two strategies, with the objective of minimizing the number of SELECT commands, hence further reducing the time overhead. Specifically, the reader executes the batch writing task bucket by bucket. For each bucket $\mathcal{B}_b$, as depicted in Fig. 7(a), we define $\mathcal{T}_b$ as the target tags belonging to bucket $\mathcal{B}_b$, i.e., $\mathcal{T}_b \triangleq \mathcal{B}_b \cap \mathcal{T}$. Our algorithm is composed of three steps.

Step 1: Construct Indication Vector (IV): We create an IV based on the CBC and DBC of the tags in $\mathcal{B}_b$, whose objective is to indicate which bit positions in the BV can be used to select target tags and deselect non-target tags. Specifically, for a bit position, if only target tags have bit 1 in this position, we term this bit position as T-homogeneous (T). Similarly we can define N-homogeneous (N) bits regarding non-target tags. The other bit positions are called heterogeneous (H). The IV is created by setting all the N bits to 0 and other bits to 1. Fig. 7(b) illustrates an example of IV with $\mathcal{T}_b = \{t_1, t_3, t_6, t_7\}$, where purple, red, and black bits are T, N, and H, respectively.

Step 2: Construct and Send SELECT Commands: Armed with the IV, the reader then exploits the functionality offered by the ACTION parameters specified in Table II to construct the SELECT commands optimally. Specifically, we term a sequence of contiguous 1's as a 1-run, with the shortest 1-run being a single bit 1. By specifying a 1-run, the reader can select the target tags mapping to the bits in the 1-run. For example, in Fig. 7(b) where $\mathcal{T}_b = \{t_1, t_3, t_6, t_7\}$, the 5th and 6th bits in the IV form a 1-run mapping to tags t_7 and t_6 . The reader can thus select t_7 and t_6 with $\text{SELECT}\{000, 5, 2, 11\}$. Recall that the reader aims at selecting all the target tags with a minimal number of SELECT commands. We develop the following greedy strategies running in iterations.

- At the beginning, all tags are marked as unselected.

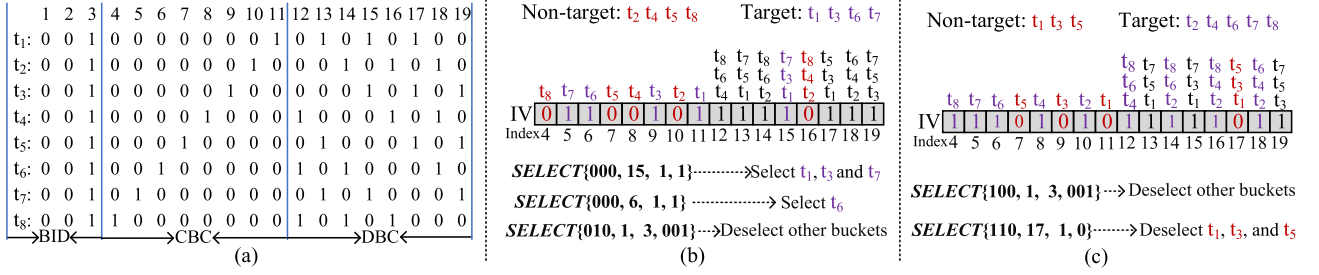


Fig. 7. (a) BVs of tags in bucket B_b ; (b) illustration of strategy based on 1-runs; (c) illustration of strategy based on 0-runs.

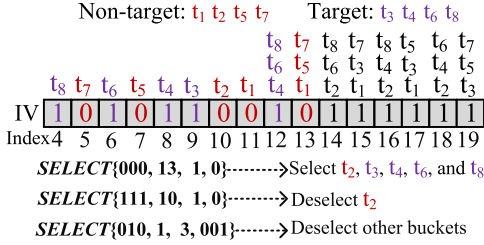


Fig. 8. Illustration of hybrid strategy.

- In each iteration, the reader identifies the 1-run covering the maximal number of unselected target tags and construct a SELECT command containing the MASK corresponding to the identified 1-run and then marks the covered tags as selected.
- The above iterations are repeated until all the target tags are selected.

In the example in Fig. 7(b), in the first iteration, the reader identifies the 15th bit in the IV as the corresponding 1-run covers a maximal number of target tags in $\mathcal{T}_b$, i.e., $\{t_1, t_3, t_7\}$. The reader issues $SELECT\{000, 15, 1, 1\}$ to select them. In the second iteration, as there is only one target tag t_6 left unselected, the reader uses the 6th bit to select it with $SELECT\{000, 6, 1, 1\}$. Finally, the reader employs $SELECT\{010, 1, 3, 001\}$ to filter out tags in other buckets if there exist tags in other buckets that are selected previously.

Symmetrically, we define 0-runs and use them to deselect non-target tags. To this end, the reader first uses the bucket ID to deselect the tags in the other buckets. It then deselects non-target tags with 0-runs. For example, in Fig. 7(c) where $\mathcal{T}_b = \{t_2, t_4, t_6, t_7, t_8\}$, the 17th bit in the IV is a degenerated 0-run mapping to the non-target tags $\{t_1, t_3, t_5\}$. The reader then deselects these tags with the command $SELECT\{110, 17, 1, 0\}$. As in the case of 1-runs, we can develop a similar greedy strategy that deselects the non-target tags with a minimal number of SELECT commands. Reconsider the example in Fig. 7(c) where $B_b - \mathcal{T}_b = \{t_1, t_3, t_5\}$, the reader first deselects the tags in the other buckets by $SELECT\{100, 1, 3, 001\}$. As the 17th bit forms a 0-run covering a maximal number of non-target tags as it covers all non-target tags in B_b , the reader issues $SELECT\{110, 17, 1, 0\}$ to deselect them, thus leaving only the target tags marked as selected.

Both of the above two strategies can achieve the goal of selecting target tags. In practice, the reader first virtually executes them and then uses the one with fewer number of SELECT commands.

Step 3: Perform Writing: As the C1G2 standard specifies that INVENTORY and ACCESS are mandatory and non-modifiable, it is impossible to reduce the delay caused by the two operations. After selecting the target tags in $\mathcal{T}_b$, the reader performs INVENTORY and ACCESS operations to write the information into them. The whole process is repeated for each bucket.

$$\begin{aligned}
 \mathbb{E}[S_b] &= \sum_{d=1}^{\lfloor \frac{t}{2} \rfloor} \sum_{w=0}^{d-1} \Pr[W_b = w \mid D_b = d] \left(\frac{w}{q} + \frac{(d-w)q}{k} \right) \\
 &+ \sum_{d=\lfloor \frac{t}{2} \rfloor + 1}^{t-1} \sum_{w=d-\lfloor \frac{t}{2} \rfloor}^{d-1} \Pr[W_b = w \mid D_b = d] \\
 &\times \left(\frac{w}{q} + \frac{(d-w)q}{k} \right) + \Pr[W_b = w \mid D_b = d] \frac{t}{q} \\
 &= \sum_{d=1}^{\lfloor \frac{t}{2} \rfloor} \sum_{w=0}^{d-1} \frac{\binom{d}{w} \binom{n-t+1}{d} \binom{t-1}{t-d}}{\binom{n}{t} \binom{t-1}{w-1}} \left(\frac{w}{q} + \frac{(d-w)q}{k} \right) \\
 &+ \frac{t \binom{n-t+1}{t}}{q \binom{n}{t}} \sum_{d=\lfloor \frac{t}{2} \rfloor + 1}^{t-1} \sum_{w=d-\lfloor \frac{t}{2} \rfloor}^{d-1} \frac{\binom{d}{w} \binom{n-t+1}{d} \binom{t-1}{t-d}}{\binom{n}{t} \binom{t-1}{w-1}} \\
 &\times \left(\frac{w}{q} + \frac{(d-w)q}{k} \right). \tag{4}
 \end{aligned}$$

C. Performance Analysis

We present our analysis on the expected number of SELECT commands of our algorithm using the strategy based on 1-runs. A similar analysis can be made for the strategy based on 0-runs. By taking the minimum of the two, we derive the time overhead for our algorithm.

Technically, we consider a single bucket B_b , and write the IV as follows

$$\underbrace{0 \cdots 0}_{z_1} \underbrace{1 \cdots 1}_{h_1} \underbrace{0 \cdots 0}_{z_2} \cdots \underbrace{0 \cdots 0}_{z_{d-1}} \underbrace{1 \cdots 1}_{h_d} \underbrace{0/1, \dots, 0/1}_{\text{variable}}$$

where z_{d-1} and h_d denote the lengths of the $(d-1)$ th 0-run and d th 1-run, respectively, and d represents the total number of

1-runs within the leftmost n bits of IV. We take Fig. 7(b) as an example to illustrate the above notation. In the example, we have $z_1 = 1, h_1 = 2, z_2 = 2, h_2 = 1, z_3 = 1$, and $h_3 = 1$. Note that the first run of IV can be a 1-run. We rank the tags by their CBC and define a *tag run* $\{t_i\}_{i=1}^j$ as a set of contiguous tags from t_i to t_j . Tag runs with a single tag and multiple tags are called *s-runs* and *m-runs*, respectively. It follows from the structure of IV that each 1-run in the leftmost N bits corresponds to a distinct tag run in $\mathcal{T}_b$, so d is the total number of tag runs in $\mathcal{T}_b$. Reconsider the example in Fig. 7(b), we have $d = 3$, indicating that $\mathcal{T}_b$ is composed of 3 tag runs $\{t_1\}$, $\{t_3\}$, and $\{t_6, t_7\}$. Assuming that tags in $\mathcal{T}$ are distributed uniformly among N , we have

$$\mathcal{T}_b = \mathcal{B}_b \cap \mathcal{T} = \frac{T}{B}, \sum_{i=1}^d h_i = \frac{T}{B}, \sum_{i=1}^{d-1} z_i = n - \frac{T}{B}.$$

Denote $t \triangleq T/B$. Given $\mathcal{T}_b$ with t tags, d then has a total of

$$\begin{aligned} & \sum_{t=1}^{\lfloor \frac{n+1}{2} \rfloor} \sum_{d=1}^t \binom{n-t+1}{d} \binom{t-1}{d-1} \\ & + \sum_{t=\lfloor \frac{n}{2} \rfloor + 1}^n \sum_{d=1}^{n-t+1} \binom{n-t+1}{d} \binom{t-1}{d-1}, \end{aligned} \quad (5)$$

distinct combinations.

Define S_b as a random variable representing the number of SELECT commands for CBC to select $\mathcal{T}_b$ from $\mathcal{B}_b$. By (5), the probability of $S_b = s$ can be calculated as

$$\Pr[S_b = s] = \begin{cases} \frac{\binom{n-t+1}{s} \binom{t-1}{s-1}}{\sum_{i=1}^t \binom{n-t+1}{i} \binom{t-1}{i-1}} & 1 \leq t \leq \lfloor \frac{n+1}{2} \rfloor \\ \frac{\binom{n-t+1}{s} \binom{t-1}{s-1}}{\sum_{i=1}^{n-t+1} \binom{n-t+1}{i} \binom{t-1}{i-1}} & \lfloor \frac{n}{2} \rfloor + 1 \leq t \leq n. \end{cases} \quad (6)$$

Next, we investigate the impact of DBC on S_b . Denote by W_b and D_b two random variables representing the numbers of s-runs and total tag runs in $\mathcal{T}_b$. Suppose that there are w s-runs in $\mathcal{T}_b$. By (6), we obtain the conditional probability

$$\Pr[W_b = w \mid D_b = d] = \frac{\binom{d}{w} \binom{n-t+1}{d} \binom{t-1}{t-d}}{\binom{n}{t} \binom{t-1}{w-1}}.$$

Recall the DBC selects any tag run via only one command, while the DBC with appropriate k and q can not only select q s-runs with only one command but also integrate multiple m-runs as one, the reader needs roughly $(w/q + (d-w)q/k)$ SELECT commands to mark $\mathcal{T}_b$ with $W_b = w$ and $D_b = d$. By combining the analysis on the impact CBC and DBC on S_b , we obtain

$$\Pr\left[S_b = \frac{w}{q} + \frac{(d-w)q}{k}\right] = \Pr[W_b = w \mid D_b = d],$$

which further leads to equality (4).

By Stirling's approximation [18], after some tedious algebraic operations, we get

$$\mathbb{E}[S_b] < \left(1 - \frac{2k - 4q + 1}{k^2/q}\right) \frac{t(n-t+1)}{n}. \quad (7)$$

With an overhead of S_b for a single bucket $\mathcal{B}_b$, we further consider the expected number of SELECT commands for B buckets. Denote S as the random variable representing the number of SELECT commands to select target tags from B buckets. We then take a similar method as for GWB to generalize S_b to S , as described in Section IV. By (7), we obtain

$$\mathbb{E}[S] < \left(1 - \frac{2k - 4q + 1}{k^2/q}\right) \frac{T(N-T+B)}{N} + \frac{T(B-1)}{N}.$$

Similarly, we can derive the number of SELECT commands for the strategy based on 0-runs as $(1 - \frac{k-q}{k^2/q}) \frac{T(N-T+B)}{N} + \frac{T(B-1)}{N}$. By taking the minimum, we obtain that the expected number of SELECT commands for our algorithm is upper-bounded by

$$\left(1 - \frac{\max\{2k-4q+1, k-q\}}{k^2/q}\right) \frac{T(N-T+B)}{N} + \frac{T(B-1)}{N}.$$

Algebraically, we can prove that the above number is minimized when $q = k/2$ and $q = k/4$, with the minimum being $0.75T(N-T+B)/N + T(B-1)/N$. Besides, to make DBC cover more combinations of discrete tags, we set $k \in [n/4, n/2]$, as $\mathcal{T}_b$ has the maximal number of combinations of discrete tags when $T = n/2$. Recall the performance of GWB derived in Section IV, we conclude that, when $B \ll N$, our algorithm incurs at least 25% performance gain.

D. Further Improvement

Our batch writing algorithm is based on the two greedy schemes exploiting the 1-runs and 0-runs in the IV by taking the one with minimal time overhead. This approach can be further improved by interlacing the two schemes. Specifically, when processing bucket $\mathcal{B}_b$, the reader first selects a set $\mathcal{T}'_b$ of tags with $\mathcal{T}_b \subset \mathcal{T}'_b$, and then deselects the tags in $\mathcal{T}'_b - \mathcal{T}_b$ such that finally only the tags in $\mathcal{T}_b$ are selected. We use Fig. 8 where $\mathcal{T}_b = \{t_3, t_4, t_6, t_8\}$ to illustrate our idea. We first use the 13th bit in the IV as a 0-run to select $\{t_2, t_3, t_4, t_6, t_8\}$. We then use the 10-bit in the IV as a 0-run to deselect t_2 . Formally, the improved algorithm is composed of two steps.

Step 1: Cover $\mathcal{T}_b$: The reader greedily chooses one or multiple 0-runs or 1-runs in the constructed IV to form one or multiple MASKS that can select a set of tags covering $\mathcal{T}_b$.

Step 2: Remove Non-Target Tags: The reader then deselects the non-target tags from the selected tags in Step 1 by exploiting the two categories of runs.

Symmetrically, we can construct another improved strategy by first deselecting a tag set covering the non-target tags and then selecting the target tags in this set.

In practice, the reader first virtually executes the above two improved strategies and then chooses the one minimizing the time overhead.

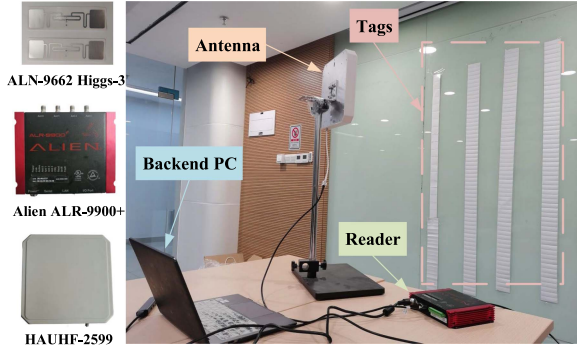


Fig. 9. Experimental setup.

VI. IMPLEMENTATION & EVALUATION

In this section, we implement our algorithm CTB using the COTS RFID reader and tag, and evaluate its empirical performance compared to the state-of-the-art algorithms EW [4], WB and GWB [5].

A. Experimental Setup and Metrics

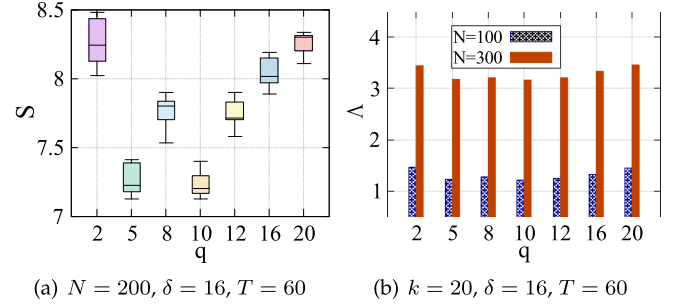
In our experiments, we have implemented an RFID system prototype as depicted in Fig. 9. A typical application scenario corresponding to our experimental setup is the the RFID systems in supermarkets or libraries, where goods attached with tags are densely distributed and the reader needs to write information to a specific category of goods.

The experimental setup and metrics are as follows.

Reader Configurations: We take Alien ALR-9900+ [17] as the RFID reader used, which is an Enterprise-class reader enabling users to deploy best-in-class C1G2 RFID solutions for object management applications. It is connected to the HAUHF-2599, a circularly polarized antenna with 9 DBi gain operating at 902-928 MHz, enabling us to implement a complete writing cycle via the bit-level mask. The number m of masks within a SELECT command is specified as 4 by the Alien reader.

Tag Settings: As demonstrated in Section V, the pre-loading operation is supported by all main-stream COTS RFID tags, such as Impinj [16] and Alien [17] tags. The time cost on this operation hinges on the space of user memory offered by diverse tags. Without loss of generality, our experiments take the ALN-9662 Higgs-3 tags [17] to evaluate algorithm performance, which provides us with 512-bit user memory, of which the first 128 bits are applied in our experiments. Specifically, the first δ bits are used for data storage while the remaining $128 - \delta$ bits for the pseudo-ID designed by each algorithm. Our system contains a total of 300 tags, which are divided into 5 buckets, each consisting of 60 tags, i.e., $N = 300$, $B = 5$ and $n = 60$.

Parameter Tuning: With a total of 128 bits user memory offered by ALN-9662 Higgs-3, δ is set to 16 or 32, thus the remaining 112 or 96 bits are used to encode tag pseudo-IDs, in which the lengths of BID, CBC, and DBC are configured as 16, 60, and 36 or 20, respectively. Hence, the number of tags that can be transacted by our CTB algorithm is up to $2^{16} \times 60 = 3,932,160$. Besides, The parameter k is then set

Fig. 10. CTB algorithm performance with diverse q .

to $n/3 = 20$ based on the theoretical analysis in Section V. For a fair comparison, we set up each algorithm using its optimal parameter configuration, and all our results are obtained from the average of 100 experimental runs.

Experimental Metrics: We trace three central performance metrics: the number of SELECT commands S , the overall execution time Λ , and the ratio R of S in our CTB algorithm to that in the other algorithms. As GWB is designed to make WB scalable to large-scale RFID systems [5], we then study WB with a small N ($N \leq 100$) while GWB under a large N ($N \leq 300$). We vary the three most important parameters q , T , and N while fixing m , k , and B , as depicted in Figs. 10, 11 and 12, to study their impact on S , Λ or R for different algorithms.

B. Experimental Results

We first discuss the impact of q on our CTB algorithm with Fig. 10, and then evaluate the impact of T or N on S or Λ with two sets of experiments, each using a distinct set of parameter setting, as illustrated in Figs. 11 and 12. As the both sets of experiments show similar trends, our subsequent analysis mainly focuses on Fig. 11 with sometimes a brief summary on Fig. 12 for comparison. Finally, we discuss the impact of T on R with Fig. 13. Note that the unit of Λ is second in the four figures.

Impact of q on S or Λ : As depicted in Fig. 10, we observe that with q varying from 2 to 20, both S and Λ show wavy trends. Specifically, when $q = 5$ or 10 , S reaches a minimal value at roughly 7.3, while 1.2 and 3.1 for Λ with $N = 100$ and 300 , respectively. This is consistent with our theoretical analysis that the delay of CTB algorithm is minimized with $q = k/2$ or $k/4$. Besides, when q varying from 10 to 20, S increases by approximately 14%, while about 17% and 10% for Λ with $N = 100$ and 300 , respectively. From Fig. 10(b), we further observe that with $N = 300$, Λ is 2.5-2.7 times of that with $N = 100$, which reveals that Λ is mainly dominated by N when T is fixed.

Impact of T on S : From Fig. 11(a), with T varying from 0 to 100, we observe that S in EW increases linearly, while that in the performance limit and other solutions increase first and then decrease, reaching the maximal value at roughly $N/2 = 50$. This is consistent with our theoretical finding that, it is more efficient to select target tags when $T \leq N/2$ or to deselect non-target ones when $T \geq N/2$. In comparison, our algorithm CTB outperforms the best existing solution WB by 30-40%,

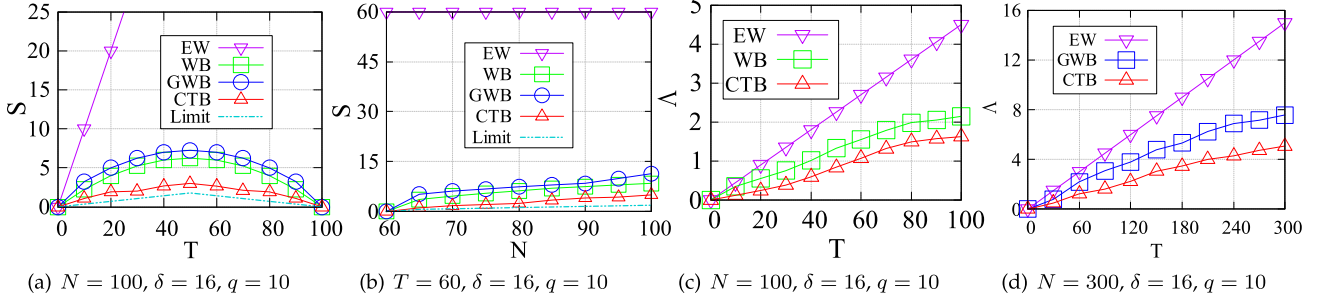


Fig. 11. Performance comparison with different parameter setting: first set of experiments.

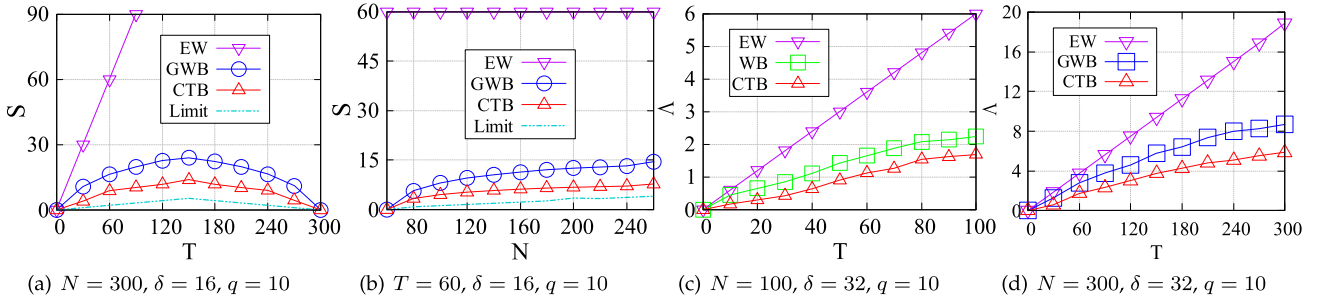
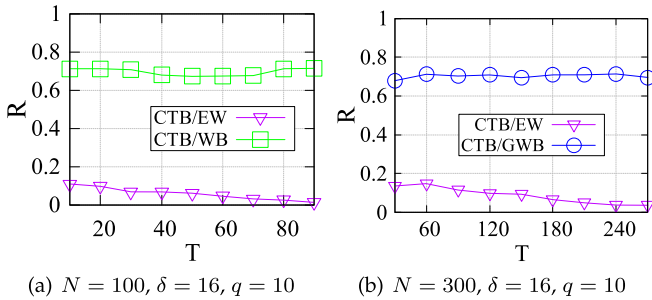


Fig. 12. Performance comparison with different parameter setting: second set of experiments.

Fig. 13. CTB algorithm versus the others under diverse T .

and narrows the performance gap between WB and the limit by roughly 60%. For instance, when $N = 50$, CTB drops S from 6 to 4 and hence reduces the delay in selecting T by 34% compared to WB, narrowing the performance gap from 4 to 1. WB slightly outperforms GWB, as GWB is based on tag grouping and hence takes extra overhead to eliminate the interference among tags in different buckets/groups. However, tag grouping enables GWB applicable to large-scale RFID systems, while WB cannot be deployed in such systems with limited user memory. By comparing Figs. 11(a) and 12(a), we further observe similar variation trend, which is also coherent with our theoretical finding and demonstrates the performance improvement of CTB over existing ones.

Impact of N on S : From Fig. 11(b), we observe that CTB significantly outperforms the other algorithms for all values of

N . The performance gap is increasing in N and goes as large as 32.6% over the best existing solution WB at $N = 100$. By comparing Figs. 11(b) with 12(b), we further observe that S in all the involved algorithms increases in N , which is consistent with our theoretical analysis. Moreover, in Fig. 12(b), with N increasing from 60 to 260, CTB incurs less performance variation in terms of S among all the involved algorithms, thus exhibiting the best performance stability. Besides, as shown in the both figures, our algorithm significantly narrows the performance gap to the limit.

Impact of T on Λ : As illustrated in Figs. 11(c), 12(c), 11(d), and 12(d), with T increasing from 0 to 300, the execution time of each algorithm increases in T . This is because more INVENTORY and ACCESS operations are needed under a larger T . Besides, we observe that the slope of EW on Λ is almost constant while that of the other algorithms is decreasing, which is coherent to our theoretical analysis again. Compared to WB and GWB, CTB reduces 25-35% execution time and incurs less performance variation in T , thus demonstrating its performance improvement and scalability.

Impact of δ on Λ : By comparing Figs. 11(d) with 12(d), we observe that the execution time of each algorithm increases in δ . For instance, under $T = 300$, the execution time of CTB algorithm is 5.1 with $\delta = 16$ while 6.2 with $\delta = 32$. Besides, the execution time of GWB is more sensitive with δ than CTB, with a fluctuation range from 4.5 to 8.5 when T varying from 120 to 300, as illustrated in Fig. 11(d). By the contrast, that for CTB ranges from 3.5 to 6.2. Moreover, Figs. 11(c) and 12(c) even demonstrate smaller performance variation on δ for CTB.

Impact of T on R : As illustrated in Fig. 13(a) and (b), with the growth of T , the performance gain of our algorithm over EW is increasing in T , while that over WB or GWB is relatively stable. Specifically, in Fig. 13(a), R for CTB/WB gets the minimal value at $T = N/2$, with a mean of roughly 0.72. We can also observe the similar variation trends on R for CTB/GWB in Fig. 13(b). This reveals that our algorithm incurs 28% performance gain over the best existing solution and obtains the maximum improvement while $T = N/2$, which is consistent with our theoretical analysis.

Overall, the following conclusions can be drawn from our experimental results: (1) T and N have a paramount influence on the number of SELECT commands needed for each algorithm, while the other parameters are less important; (2) S increases linearly in T in EW, while first increases and then decreases, reaching the maximal value at $T = N/2$ in other algorithms; (3) S is increasing and concave in N ; (4) Our algorithm CTB outperforms all the involved algorithms with non-negligible performance gain in terms of time overhead, scalability and stability, and significantly narrows the performance gap between practice and theory. These conclusions are consistent with our theoretical results as shown in Table III.

VII. RELATED WORKS

Reading and writing are two fundamental functionalities in RFID systems. The former is to collect required information from a given subset of tags in the system, while the latter aims at writing a piece of information into them.

In RFID systems, major existing algorithms [19], [20], [21], [22], [23], [24], [25], [26] are dedicated to enhancing the read throughput from tags to the reader, which are generally divided into two categories. The first employs Aloha protocol or tree structure to collect tag IDs one by one. Technically, the Aloha-based algorithms [19] carry out a slotted-frame to attribute a singleton slot to each tag such that they can report their IDs without interference. The tree-based algorithms [20] then employs tags' distinct ID prefixes to repetitively interrogate them until all of them are identified. By contrast, the second category tends to implement a palette of specified functions involving cardinality estimation [21], [22], anomaly event detection [1], [4], target tag search [23], [24], tag sampling [25], [26] etc., by gathering and then analyzing functional tag data.

Regarding batch writing, there exist a handful of solutions in the literature [4], [5], [27], [28], [29]. However, most of them are not compatible with the C1G2 standard, hence cannot be deployed in COTS RFID systems, thus significantly limiting their practicality. To the best of our knowledge, there are only three existing batch writing algorithms implementable in COTS RFID systems, exclusive writing (EW) [4], write bundling (WB) [5], and generalized write bundling (GWB) [5]. EW [4] performs batch writing by writing the information to target tags one by one. This approach can be deployed in COTS tags, with the limitation of being time-consuming. To improve the time efficiency, WB [5] bundles multiple tags to form a single batch, thus reducing the number of SELECT commands. To this end, it relies on a bit vector pre-loaded to each tag. To fit the bit vector to

tag memory, GWB [5] further divides tags into multiple groups and performs batch writing group by group.

Compared with the state-of-the-art solutions WB and GWB, our batch writing algorithm further reduces the time overhead with two novel design techniques. First, we propose a novel bit vector design by integrating both CBC and DBC, enabling the reader to aggregate both contiguous and discrete targets. Second, we further optimize the batch writing sequence by exploiting the eight ACTION parameters specified in the SELECT command to make an optimal decision among selecting target tags and deselecting the non-target tags, and further combining the two strategies, hence further reducing the time overhead. As a result, our batch writing algorithm outperforms EW, WB and GWB in time overhead, as shown in Table III.

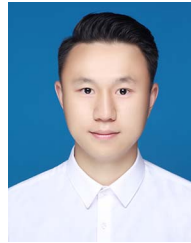
VIII. CONCLUSION

In this paper, we have investigated the batch writing problem in COTS RFID systems. Theoretically, we have established the performance limit for any batch writing algorithm. We have then used the derived bound to gauge the state-of-the-art solutions, notably WB and GWB. Algorithmically, we have developed a novel batch writing algorithm by leveraging an efficient and compact encoding scheme for tag pseudo-ID and a writing sequence optimization strategy. We have formally proved that our algorithm outperforms the best state-of-the-art solution by at least 25%. Regarding future work, the first item in our research agenda is to see whether it is possible to reduce the performance gap between our algorithm and the performance limit, by either improving our algorithm or deriving tighter performance bound. Our second plan is to extend our algorithmic framework to other tag monitoring tasks involving batch writing for COTS RFID systems.

REFERENCES

- [1] K. Liu, L. Chen, J. Huang, S. Liu, and J. Yu, "Revisiting RFID missing tag identification," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 710–719.
- [2] X. Liu, D. Liu, J. Zhang, T. Gu, and K. Li, "RFID and camera fusion for recognition of human-object interactions," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, 2021, pp. 296–308.
- [3] D. Wu, L. Lu, M. J. Hussain, S. Li, M. Li, and F. Zhang, "R3: Reliable over-the-air reprogramming on computational RFIDs," *ACM Trans. Embedded Comput. Syst.*, vol. 17, Sep. 2017, Art. no. 9.
- [4] T. Li, S. Chen, and Y. Ling, "Identifying the missing tags in a large RFID system," in *Proc. 11th ACM Int. Symp. Mobile Ad Hoc Netw. Comput.*, 2010, pp. 1–10.
- [5] J. Liu, X. Chen, X. Liu, X. Zhang, X. Wang, and L. Chen, "On improving write throughput in commodity RFID systems," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1522–1530.
- [6] EPC global, "EPC Radio-Frequency Identity Protocols generation-2 UHF RFID Standard GS1," 2018, Accessed: Jun. 8, 2023. [Online]. Available: https://www.gs1.org/sites/default/files/docs/epc/gs1-epc-gen2v2-uhf-airinterface_i21_r_2018-09-04.pdf.
- [7] S. A. Cook, "The complexity of theorem-proving procedures," in *Proc. 3rd Annu. ACM Symp. Theory Comput.*, 1971, pp. 151–158.
- [8] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*. Cambridge, U.K.: Cambridge Univ. Press, 2012.
- [9] D. P. Woodruff and Q. Zhang, "An optimal lower bound for distinct elements in the message passing model," in *Proc. 25th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2014, pp. 718–733.
- [10] A. C.-C. Yao, "Some complexity questions related to distributive computing," in *Proc. 11th Annu. ACM Symp. Theory Comput.*, 1979, pp. 209–213.

- [11] S. Ross, "A first course in probability, sixth edition," in *Differential Geometry*, Hoboken, NJ, USA: Wiley, 2002.
- [12] A. Wald, "On cumulative sums of random variables," *Ann. Math. Statist.*, vol. 15, no. 3, pp. 283–296, 1944.
- [13] N. Etemadi, "An elementary proof of the strong law of large numbers," *Zeitschrift Für Wahrscheinlichkeitstheorie Und Verwandte Gebiete*, vol. 55, no. 1, pp. 119–122, 1981.
- [14] J. Liu, X. Chen, H. Liu, H. Gong, Y. Wang, and L. Chen, "Time-efficient range detection in commodity RFID systems," in *Proc. IEEE 28th Int. Conf. Netw. Protoc.*, 2020, pp. 1–10.
- [15] X. Yu, J. Liu, S. Zhang, X. Chen, X. Zhang, and L. Chen, "Encoding-based range detection in commodity RFID systems," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 680–689.
- [16] Impinj inc., Accessed: Jun. 8, 2023. [Online]. Available: <https://www.impinj.com>
- [17] Alien, "Alien ALR-9900+ Files," 2022, Accessed: Jun. 8, 2023. [Online]. Available: <https://www.alientechnology.com/products/files-2/alr-9900/>
- [18] D. Romik, "Stirling's approximation for $n!$: The ultimate short proof?," *Amer. Math. Monthly*, vol. 107, pp. 556–557, 2000.
- [19] J. Su, A. X. Liu, Z. Sheng, and Y. Chen, "A partitioning approach to RFID identification," *IEEE/ACM Trans. Netw.*, vol. 28, no. 5, pp. 2160–2173, Oct. 2020.
- [20] Y.-C. Lai, L.-Y. Hsiao, H.-J. Chen, C.-N. Lai, and J.-W. Lin, "A novel query tree protocol with bit tracking in RFID tag identification," *IEEE Trans. Mobile Comput.*, vol. 12, no. 10, pp. 2063–2075, Oct. 2013.
- [21] Y. Zheng and M. Li, "PET: Probabilistic estimating tree for large-scale RFID estimation," *IEEE Trans. Mobile Comput.*, vol. 11, no. 11, pp. 1763–1774, Nov. 2012.
- [22] Z. Zhou, B. Chen, and H. Yu, "Understanding RFID counting protocols," *IEEE/ACM Trans. Netw.*, vol. 24, no. 1, pp. 312–327, Feb. 2016.
- [23] J. Yu, W. Gong, J. Liu, L. Chen, and K. Wang, "On efficient tree-based tag search in large-scale RFID systems," *IEEE/ACM Trans. Netw.*, vol. 27, no. 1, pp. 42–55, Feb. 2019.
- [24] F. Zhu, B. Xiao, J. Liu, X. Liu, and L.-J. Chen, "PLAT: A physical-layer tag searching protocol in large RFID systems," in *Proc. IEEE 13th Annu. Int. Conf. Sens. Commun. Netw.*, 2016, pp. 1–9.
- [25] X. Xie et al., "Implementation of differential tag sampling for COTS RFID systems," *IEEE Trans. Mobile Comput.*, vol. 19, no. 8, pp. 1848–1861, Aug. 2020.
- [26] J. Liu, S. Chen, Q. Xiao, M. Chen, B. Xiao, and L. Chen, "Efficient information sampling in multi-category RFID systems," *IEEE/ACM Trans. Netw.*, vol. 27, no. 1, pp. 159–172, Feb. 2019.
- [27] J. Liu, B. Xiao, S. Chen, F. Zhu, and L. Chen, "Fast RFID grouping protocols," in *Proc. IEEE Conf. Comput. Commun.*, 2015, pp. 1948–1956.
- [28] F. Zhu, B. Xiao, J. Liu, Y. Wang, and L. Chen, "Dynamic grouping in RFID systems," in *Proc. IEEE 14th Annu. Int. Conf. Sens. Commun. Netw.*, 2017, pp. 1–9.
- [29] J. Yu, J. Liu, L. Chen, and Y. Zhu, "Efficient group labeling for multi-group RFID systems," in *Proc. IEEE/ACM 25th Int. Symp. Qual. Service*, 2017, pp. 1–2.



Kanghuai Liu received the BE degree in software engineering from Nanchang University, China, in 2017, and the MS degree in software engineering from Central South University, China, in 2020. He is currently working toward the PhD degree in computer science and technology with the School of Computer Science and Engineering, Sun Yat-sen University, China. His main research interests include distributed algorithm design, radio frequency identification (RFID), wireless backscatter communication, packet classification, and route origin authorization (ROA).



Lin Chen (Member, IEEE) received the BE degree in radio engineering from Southeast University, China, in 2002, the MS degree in networking from the University of Paris 6, and the engineer diploma and PhD degrees from Telecom ParisTech, Paris, in 2005 and 2008, respectively. He is currently a professor with the School of Computer Science and Engineering, Sun Yat-sen University. His main research interests include distributed algorithm design and analysis in networked systems.



Jihong Yu (Member, IEEE) received the BE and ME degrees from the Chongqing University of Posts and Telecommunications, Chongqing, China, in 2010 and 2013, respectively, and the PhD degree in computer science from the University of ParisSud, Orsay, France, in 2016. He was a post-doctoral researcher with the School of Computing Science, Simon Fraser University, Canada. He is currently a professor with the School of Information and Electronics, Beijing Institute of Technology. His research interests include RFID, backscatter networking, and Internet of Things.



Haochen Cui received the BE degree in software engineering from the Dalian University of Technology, China, in 2022. He is currently working toward the postgraduate degree in computer science and technology with the School of Computer Science and Engineering, Sun Yat-sen University, China. His main research interests include backscatter communication and radio frequency identification (RFID) technology.